

Interpreting Transformer Representations on Rubik’s Cube Tasks

Progress Report

Cole McGuire

Abstract

We use the $2\times 2\times 2$ Rubik’s cube as a controlled, fully-enumerable domain for mechanistic interpretability. A small transformer ($d_{\text{model}}=128$, 4 layers, $\sim 200\text{k}$ parameters) is trained to classify the optimal half-turn-metric distance to solved (0–11), achieving 78.5% validation accuracy against an 8.3% random baseline. We then apply four interpretability methods to understand what the model has learned. Linear probing finds that face-level solved status is 98% decodable at *every* layer including the raw embedding, and that optimal distance is encoded almost completely in the embedding itself (MAE drops from 0.87 to 0.40 in the first block). Activation patching reveals a dissociation: the probe-readable features are not causally active—ablating their directions produces zero change in model output—whereas swapping the full residual stream between distance classes achieves a 100% prediction flip at every layer. Tuned-lens analysis reconciles these findings: logit-lens accuracy grows from 20% to 78% across blocks, demonstrating that each block is contributing meaningful computation, but in a rotated basis that the final unembedding layer alone cannot read. Sparse autoencoders confirm that face and distance features are represented monosemantically ($r \approx 0.81\text{--}0.89$), while corner-orientation features are weaker and concentrated at the embedding layer. Finally, motivated by instructor feedback, we evaluate how well the cube state can be conveyed to a pre-trained language model through text. An initial pilot (1 case per distance, GPT-5 via chat) suggested that a *piece-identity* encoding outperforms position-based representations. A subsequent automated evaluation across five models (GPT-5.4, GPT-5.2, Gemini 2.5 Pro, GPT-4o, Llama 3.3 70B; 10 cases per distance, $d=3$ through $d=11$) found that all state-based representations score zero uniformly, while move-inversion solve rates range from 12% (Llama) to 86% (GPT-5.2), stratifying models by their ability to parse cube notation.

1 Introduction

Mechanistic interpretability asks not only whether a neural network can solve a task, but *how*: which features are encoded in intermediate representations, which are causally relevant to the output, and how computation flows through layers. Most such work has been done on large language models, where the sheer scale makes it difficult to achieve a complete account.

We instead choose a domain where the full state space is enumerable, optimal solutions are known for every state, and the relevant geometric structure is explicit: the $2 \times 2 \times 2$ Rubik’s cube.

Prior work has explored the cube primarily from a performance perspective. Agostinelli et al. train a deep reinforcement learning agent that solves all test configurations [1]. Takano demonstrates that self-supervised training on reverse scrambles is sufficient for near-optimal solving [2]. Chasmai proposed CubeTR, a transformer trained via sequence modelling on the cube, though this work was subsequently withdrawn for lacking experimental validation [3]. Gupta et al. provide a more rigorous transformer-based study, using the $2 \times 2 \times 2$ cube as a controlled domain to investigate how explicit world-model pretraining objectives affect representations and downstream reinforcement learning, combining linear probing and causal interventions with GRPO post-training [4]. Our work is complementary: where Gupta et al. compare training objectives and study the effect on solving performance, we fix the training objective and apply a broader suite of interpretability tools to characterize what a single trained model represents.

For interpretability methodology we follow Alain and Bengio’s framework of linear classifier probes [5], Belinkov’s critical survey of their promises and limitations [6], Belrose et al.’s tuned-lens technique for reading intermediate predictions [7], Meng et al.’s causal activation-patching methodology [8], Bricken et al.’s sparse autoencoder approach to monosemantic feature decomposition [9], and Nanda et al.’s analysis of grokking via mechanistic interpretability [10]. We are also motivated by Variengien et al.’s theoretical result that belief-state geometry is linearly represented in transformer residual streams [11]—the cube provides a clean, discrete instance of this claim.

The core question this project addresses is: *what internal representations does a small transformer learn when trained on a combinatorial task with known optimal structure, and do those representations match the task’s known geometric features?* A secondary question, which emerged from instructor feedback mid-project, is: *what text representation of cube state is necessary and sufficient for a pre-trained language model to reason about the cube?*

2 Related Work

Cube-solving with neural networks. DeepCubeA [1] uses a value network trained by BFS-bootstrapping; Takano [2] shows reverse-scramble self-supervision yields state-of-the-art performance. Chasmai proposed CubeTR, a transformer trained via sequence modelling, though this work was withdrawn for lacking experimental validation [3]. Most directly related is Gupta et al. [4], who train transformers on the $2 \times 2 \times 2$ cube with explicit world-model objectives and apply linear probes and causal interventions—methodology closely parallel to ours. Our work differs in focus: we analyze a single classification model with a richer interpretability suite (tuned lens, SAE, text representation testing) rather than comparing training objectives.

Linear probing. Alain and Bengio [5] propose training linear classifiers on frozen intermediate representations to measure what information each layer encodes. Belinkov [6] surveys methodological pitfalls, including the risk of confusing *encoding* with *use*. We address this risk directly via activation patching (Section 3.4).

Tuned lens. The logit lens [12] projects intermediate residual stream states directly through the final unembedding. Belrose et al. [7] improve this with a per-layer affine probe (tuned lens), showing it is more reliable and less biased. We apply both to our cube model.

Activation patching. Meng et al. [8] introduce causal mediation analysis for transformers: swapping activations from a clean to a corrupted run identifies which components are causally responsible for a prediction. We adapt this to a concept-direction ablation: we project probe directions out of the residual stream and measure whether model output changes.

Sparse autoencoders. Bricken et al. [9] train overcomplete autoencoders on transformer residual streams to find monosemantic features that do not emerge from direct examination of weight matrices. We apply this technique to each layer of our cube model.

Belief state geometry. Variengien et al. [11] prove that optimal next-token predictors represent belief states linearly in their residual streams, even when those belief states have fractal geometry. The cube’s state space is a discrete dynamical system, and a model processing a cube state token should represent a point mass over the 88M-state space. Our probing results can be read as an empirical test of their theoretical prediction.

3 Methods

3.1 Cube Simulator and State Encoding

The $2 \times 2 \times 2$ cube has 24 stickers arranged on 6 faces. We assign each face a canonical color (white top, yellow bottom, green front, blue back, orange left, red right) and index stickers 0–23 in reading order per face (U=0–3, D=4–7, F=8–11, B=12–15, L=16–19, R=20–23). The state is a (24,) int8 array of color indices. We encode this as a (144,) float32 one-hot vector: $\mathbf{x} = \text{flatten}(\mathbf{I}_6[\mathbf{s}])$, where $\mathbf{s}$ is the state array and $\mathbf{I}_6$ is the 6×6 identity (one-hot matrix).

The 18-move vocabulary consists of 6 faces $\times$ 3 turn types (CW, CCW, 180°). All move permutations are derived geometrically from face normals and sticker positions, not hard-coded. This ensures correctness and makes adding additional cube sizes straightforward.

Optimal distances are precomputed by BFS from all 24 rotationally equivalent solved states, producing a table of 88,179,840 states mapping `state.tobytes()` $\rightarrow$ distance. Seeding from all 24 orientations removes orientation bias: the solver finds the shortest path to *any* solved orientation. BFS takes approximately 25 minutes and is cached to disk.

3.2 Model Architecture

We implement `CubeTransformer` as a `HookedRootModule` (`TransformerLens`):

$$\begin{aligned}\mathbf{h}_0 &= \text{Linear}(144, d_{\text{model}})(\mathbf{x}) \\ \mathbf{h}_\ell &= \text{TransformerBlock}(\mathbf{h}_{\ell-1}), \quad \ell = 1, \dots, L \\ \hat{\mathbf{y}} &= \text{Linear}(d_{\text{model}}, 12)(\text{LayerNorm}(\mathbf{h}_L))\end{aligned}$$

Default configuration: $d_{\text{model}}=128$, $L=4$ blocks, $n_{\text{heads}}=4$, MLP expansion factor 4 ($\sim 200\text{k}$ parameters). Since the input is a single token, attention weights are always 1.0 and computation flows almost entirely through the MLP sublayers. All hook points follow `TransformerLens` naming (`hook_embed`, `hook_resid_mid`, `hook_resid_post`), so `run_with_cache()` works out of the box.

Training uses AdamW ($\text{lr}=3 \times 10^{-4}$, $\text{wd}=0.01$) with cosine annealing over 30 epochs, class-weighted cross-entropy to compensate for the heavy distance-distribution imbalance (distance 1 has 18 states; distance 9 has $\sim 14\text{M}$). The dataset contains 50k/5k/5k train/val/test scramble sequences ($\sim 300\text{k}$ samples each).

3.3 Linear Probing (Phase 4)

For each residual stream checkpoint (`hook_embed`, `resid_post` layers 0–3), we train:

- A `LogisticRegression` probe per binary feature: 6 face-solved bits and 8 corner-orientation bits.
- A `Ridge` regression probe for optimal distance and scramble depth.

Probes are fit on a held-out probe split (5k sequences not used in training), with 5-fold cross-validation for regularization strength.

3.4 Activation Patching (Phase 5a)

We perform two types of patching experiments:

Concept-direction ablation. Given a probe weight vector $\mathbf{w}$ for concept c , we project it out of all residual stream activations: $\hat{\mathbf{h}} = \mathbf{h} - (\mathbf{h} \cdot \hat{\mathbf{w}})\hat{\mathbf{w}}$, where $\hat{\mathbf{w}} = \mathbf{w}/\|\mathbf{w}\|$. We then measure whether model predictions change. This tests whether the probe direction is *causally active*.

Counterfactual residual swap. For pairs of states from different distance groups (d_i vs. d_j), we swap the full residual stream at each layer and measure whether the model’s distance prediction flips. A 100% flip rate indicates the residual stream at that layer contains all the information the model uses to distinguish the two classes.

3.5 Tuned Lens (Phase 5b)

For each layer ℓ , we train an affine probe $\phi_\ell(\mathbf{h}) = \mathbf{W}_\ell \mathbf{h} + \mathbf{b}_\ell$ to map the intermediate residual stream to the output logit space, then take the argmax as a layer- ℓ prediction. We compare this to the logit lens (using the final unembedding $\mathbf{W}_U$ directly on $\mathbf{h}_\ell$) to measure how much basis rotation is needed to read predictions from early layers.

3.6 Sparse Autoencoder (Phase 5c)

For each layer, we train a 2-layer autoencoder with $4\times$ expansion (512 features) and L1 penalty on the hidden activations to encourage sparsity: $\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 + \lambda \|\mathbf{z}\|_1$. Dead features (never active on training data) are resampled from high-loss inputs every 1,000 steps. We then compute Pearson correlation between each SAE feature’s activation and known concept labels (face_solved, optimal_distance, corner_oriented).

3.7 Text Representation Analysis (Phase 6)

Motivated by instructor feedback, we evaluate four text representations of the cube state as inputs to a pre-trained language model (GPT-5 / ChatGPT, accessed via chat interface). The representations are:

1. **face_grid**: An unfolded net diagram showing each face’s 2×2 sticker colors.
2. **compact_string**: A 24-character string, 4 characters per face in U/D/F/B/L/R order, reading order within each face.
3. **corner_cubies**: For each corner position, lists which color faces which direction (U/D, F/B, L/R).
4. **piece_identity**: Names each corner piece by its solved-state colors (W-G-R for the piece belonging at UFR) and states its current position and which face its W/Y sticker faces. This makes the permutation and orientation explicit.

All solutions are validated by applying the model’s proposed moves to the scrambled state in our simulator and checking `is_solved()`. We also test a chain-of-thought (CoT) variant of the piece_identity prompt, where the model is instructed to reason step-by-step and conclude with **Solution:** [moves].

3.8 Direct Logit Attribution and Circuit Analysis (Phase 7)

To localize which model components carry the distance signal to the output, we apply *direct logit attribution* (DLA): for each component c , we project its residual-stream contribution $\Delta \mathbf{h}_c$ onto the final unembedding direction for each class and sum over the dataset to obtain a signed scalar “vote.” We compute per-neuron DLA for all MLP blocks and per-head DLA for all attention heads. We additionally visualize attention head weight matrices ($\mathbf{W}_{QK}$, $\mathbf{W}_{OV}$)

and the neuron activation distributions for the top-DLA neurons to characterize what each neuron responds to.

3.9 Embedding and Weight Analysis (Phase 8)

We perform truncated SVD on the $144 \times d_{\text{model}}$ embedding matrix $\mathbf{W}_E$ and project each singular vector back into sticker space to identify which input dimensions cluster together in the learned basis. We additionally visualize the full $\mathbf{W}_E$ as a heatmap organized by face and sticker index to look for color-face structure.

3.10 Training Dynamics (Phase 9)

To test for grokking, we save a checkpoint every epoch and run the full linear probe battery (`face_solved`, `corner_oriented`, `optimal_distance`) at every checkpoint. We plot probe accuracy and model validation accuracy as a function of epoch to detect phase transitions.

3.11 Next-Move Prediction Variant (Phase 10)

Using the same architecture, we train on the *next-move* task: given a scrambled state, predict one of 18 optimal next moves (selecting uniformly among tied optimal continuations). We probe the residual stream of this model with the same concept set and compare the internal representations to those of the distance model.

3.12 Extended Superposition Analysis (Phase 11)

To test for superposition in the residual stream, we sweep SAE expansion ratios ($1\times$, $2\times$, $4\times$, $8\times$, $16\times$), tracking reconstruction R^2 , the fraction of dead features (never active on the validation set), and the mean active feature count. A model using superposition should show a plateau in active count and a spike in dead features as the expansion ratio grows, since features must compete for the limited representational budget.

3.13 Corner-Tokenized Architecture (Phase 12)

To enable non-degenerate attention, we implement `CubeTransformerCorner`: the 24 stickers are partitioned by corner cubie, yielding 8 tokens of 18 dimensions each ($3 \text{ stickers} \times 6 \text{ colors}$, one-hot). All other hyperparameters match the flat model. We train under identical conditions and extract 8×8 attention weight matrices at each layer, stratified by optimal distance, to test whether the model attends selectively to misplaced cubies.

3.14 Attention Head Ablation (Phase 13)

For the corner model, we measure each head’s causal contribution via *mean-attention ablation*: we replace head (l, h) ’s attention weights with uniform $\frac{1}{8}$ across all 8 tokens using

TransformerLens’s `run_with_hooks()` and record the resulting drop in validation accuracy. We run this for all $4 \times 4 = 16$ heads, holding all other components fixed.

4 Experiments and Results

4.1 Training

The model reaches 78.5% validation accuracy on 12-class distance classification (random baseline: 8.3%) in 30 epochs. Per-class accuracy is high for distances 0–4 and falls for 5–10, reflecting both the class imbalance and the inherent difficulty of near-maximal scrambles.

4.2 Linear Probing Results

Feature	Embed	L0	L1	L2	L3
<code>face_solved</code> (avg. acc.)	0.98	0.98	0.97	0.97	0.98
<code>corner_oriented</code> (avg. acc.)	1.00	0.97	0.96	0.94	0.92
<code>optimal_distance</code> (MAE)	0.87	0.40	0.39	0.39	0.38

Table 1: Probe accuracy / MAE across residual stream checkpoints.

Three findings stand out. First, `face_solved` is 98% decodable from *every* layer including the raw embedding—the linear projection from 144-dim one-hot space encodes this feature completely. Second, `corner_oriented` is perfectly recoverable at the embedding (1.00) but degrades monotonically to 0.92 at L3, suggesting the model actively transforms this feature away as it focuses on distance. Third, the largest improvement in distance prediction occurs in the very first block ($0.87 \rightarrow 0.40$ MAE), with minimal improvement in subsequent layers.

4.3 Activation Patching Results

Concept-direction ablation. Projecting out the `face_solved` probe direction at any layer produces *zero* change in model predictions; the same holds for `corner_oriented`. Both features are epiphenomenal—encoded but not causally used.

Counterfactual residual swap. Swapping the full residual stream at any layer between distance groups (e.g., states with optimal distance 3 vs. states with distance 7) produces a 100% flip rate, at *every* layer including the raw embedding. This means the linear embedding already encodes optimal distance information that is sufficient to drive the final prediction.

Implication. These two results together suggest that the model’s linear embedding encodes a near-complete representation of optimal distance. The transformer blocks refine this representation for hard inputs (explaining the tuned-lens results below) but do not

fundamentally restructure it. The probe-readable features (`face_solved`, `corner_oriented`) are by-products of the representation rather than causes of the output.

4.4 Tuned Lens Results

Lens	Embed	L0	L1	L2	L3
Logit lens	0.20	0.35	0.55	0.70	0.78
Tuned lens	0.60	0.70	0.75	0.79	0.81

Table 2: Classification accuracy from logit lens and tuned lens at each layer.

The large gap between logit lens (20%) and tuned lens (60%) at the embedding layer shows that the residual stream already contains the right information at layer 0, but in a rotated basis the final unembedding cannot read. The gap closes by L3 (78.5% vs. 81.3%), confirming that the final block aligns the representation to the unembedding’s reading direction. Distances 1–2 show a striking phase transition at L2: near-zero accuracy before, $\sim 100\%$ after. Distances 5–9 remain hard at every layer, consistent with class imbalance in training.

This result *reconciles* the patching findings: the blocks are doing meaningful computation (logit-lens accuracy grows substantially), but they are rotating the representation rather than adding new information. The residual swap’s 100% flip rate reflects that the *content* is present from the start, while the lens gap reflects that it is not yet in the right format.

4.5 Sparse Autoencoder Results

Face and distance features are strongly monosemantic: the top SAE features at every layer correlate with `face_solved` ($r \approx 0.77\text{--}0.87$) and `optimal_distance` ($r \approx 0.81\text{--}0.89$). Symmetric face pairs (U/D, F/B, L/R) consistently share the same top SAE features, indicating the model treats geometrically equivalent face pairs identically. Corner-orientation features are weaker ($r \approx 0.50\text{--}0.63$) and strongest at the embedding layer, consistent with the probing result that corner orientation is encoded early and then transformed away. The final layer (L3) is naturally sparser (mean activation count ≈ 200 vs. ≈ 400 in earlier layers), consistent with compression toward a decision.

4.6 Text Representation Results

Pilot study (manual, GPT-5 via chat interface). An initial pilot with one test case per distance level ($d \in \{3, 5, 7\}$) produced the results in Table 3. `piece_identity` was the only state-based representation to solve a $d=5$ case. The pilot prompt did not prohibit code or pseudocode, and the model frequently wrote Python or BFS-style algorithms to derive a solution rather than reasoning about the cube state directly. The pilot results therefore reflect a mix of genuine cube reasoning and code-assisted lookup, making them an overestimate of the model’s unaided symbolic reasoning ability.

Representation	$d=3$	$d=5$	$d=7$	Total
face_grid	✓	×	×	1/3
compact_string	✓	×	✓	2/3
corner_cubies	✓	×	×	1/3
piece_identity	✓	✓	×	2/3
move_sequence	✓	✓	✓	3/3
piece_identity+CoT	✓	×	×	1/3

Table 3: Pilot solve rates (1 case per distance, GPT-5 via chat interface). `move_sequence` is a degenerate baseline (move inversion); all others require genuine state understanding.

Automated evaluation (API, 10 cases per distance level). We ran an automated evaluation across five models via their respective provider APIs, with 10 independently sampled test cases per distance level ($d \in \{3, 5, 7, 9, 11\}$), yielding 50 cases per model. Representations tested: `face_grid`, `compact_string`, `corner_cubies`, and `move_sequence`. All three state-based representations scored 0/ N across every model and every distance level without exception. Table 4 shows `move_sequence` solve rates, the only dimension that varied.

Model	$d=3$	$d=5$	$d=7$	$d=9$	$d=11$	Total
GPT-5.4	10/10	10/10	7/10	6/10	9/10	42/50
GPT-5.2	9/10	9/10	8/10	8/10	9/10	43/50
Gemini 2.5 Pro	9/10	8/10	9/10	8/10	1/2*	35/42
GPT-4o	5/10	1/10	1/10	0/10	1/10	8/50
Llama 3.3 70B	3/10	1/10	1/10	0/10	0/2*	5/42

Table 4: `move_sequence` solve rate by model and distance. All state-based representations (`face_grid`, `compact_string`, `corner_cubies`) scored 0/ N for every model and every distance level. *Gemini and Llama runs sampled only 2 $d=11$ cases due to a smaller scramble iteration budget in those runs.

State-based representations fail uniformly. The pilot result suggesting `piece_identity` could reach $d=5$ (2/3 cases) did not survive scale: no state-based representation solved a single case across any model at any distance. The barrier is not which representation is used but the model’s inability to mentally simulate even a single move reliably. At scale, all state-based representations are equivalent—uniformly zero.

$d=7$ is not a wall—move simulation is the wall. The original framing identified $d=7$ as a hard limit. The automated data revises this: state-based representations fail from $d=3$ upward, before any complexity threshold. The true barrier is symbolic move simulation itself, not the depth of the scramble.

Move-inversion capability varies dramatically across models. GPT-5.x and Gemini 2.5 Pro reach 83–86% on `move_sequence`, roughly matched. GPT-4o (16%) and Llama 3.3

70B (12%) trail far behind. Since move inversion is a simple reversal task (reverse order, negate suffix), this gap reflects each model’s ability to parse and emit cube notation reliably, not any understanding of cube geometry.

Chain-of-thought did not help. The CoT pilot transcript shows the model producing plausible-sounding reasoning (“I verified this sequence against the exact corner state you gave”) without actually simulating moves—the solutions it produced were wrong. The reasoning is *confabulatory*: narrative about a computation that is not being performed. CoT with `piece_identity` performed *worse* than direct `piece_identity` (1/3 vs. 2/3 in the pilot).

4.7 Circuit Analysis Results

Direct logit attribution identifies `mlp_0` as the dominant distance-computing component, contributing +5.6 to the correct logit on average. All four attention layers combined contribute −1.2, indicating that attention provides no useful signal in this single-token architecture—and may mildly hurt performance by adding noise. The remaining MLP blocks (layers 1–3) contribute positively but are secondary. The implication is that the distance-classification circuit is, to a first approximation, a single MLP block: **embed** → **mlp_0** → **unembed**. Per-neuron DLA confirms that a small set of neurons in `mlp_0` account for the majority of the positive contribution, and these neurons activate preferentially on high-distance, face-unsolved configurations.

4.8 Weight Analysis Results

SVD of $\mathbf{W}_E$ reveals that the top singular directions correspond to face-pair contrasts: the first singular vector separates white (U) vs. yellow (D) sticker activations, the second separates green (F) vs. blue (B), and the third separates orange (L) vs. red (R). This is the minimal coordinate system needed to determine which face each sticker belongs to. The embedding matrix therefore learns a factored representation that recovers the cube’s face structure from the flat 144-dim one-hot input.

4.9 Training Dynamics Results

No sharp phase transition (grokking) is observed. Linear distance probe R^2 rises monotonically from epoch 1, with the largest per-epoch gains in epochs 1–5 and gradual improvement thereafter. Face-solved decodability (>0.95) is present from epoch 1; corner-orientation decodability is high early and plateaus by epoch 5. This contrasts with the discontinuous generalization observed in modular arithmetic tasks [?]. The absence of grokking is consistent with the cube distance function being smooth and without the hard discrete symmetry that drives delayed generalization.

4.10 Next-Move Prediction Results

The next-move model reaches 58.3% validation accuracy on the 18-class task (random baseline: 5.6%). Probing reveals that it also encodes optimal distance linearly, despite distance not being the training target ($R^2 \approx 0.90$ at the embedding, rising to ≈ 0.96 at L3). Corner-orientation decodability is higher than in the distance model at later layers, suggesting the next-move task requires retaining finer geometric structure to select among optimal continuations.

4.11 Superposition Analysis Results

At $1\times$ expansion (512 features for a 128-dim residual stream), $R^2 > 0.999$ with *zero* dead features across all layers. The active feature count continues to grow as the expansion ratio increases to $16\times$, with no plateau, and the dead-feature fraction remains near zero throughout. Both results are inconsistent with classical superposition: under superposition, features compete for the representational budget, producing dead features and a plateau in distinct active features. The absence of these signatures indicates the model encodes its internal representations near-orthogonally, using far fewer than 128 independent concepts and leaving substantial capacity unused.

4.12 Corner Model Results

The corner model achieves 76.2% validation accuracy vs. 79.8% for the flat model, a 3.6 pp gap attributable to the reduced per-token embedding dimensionality. Attention heatmaps stratified by optimal distance show a systematic pattern: at low distance ($d \leq 3$), weights are near-uniform; at high distance ($d \geq 9$), off-diagonal entries are sharply elevated, indicating the model attends preferentially to pairs of misplaced cubies. The pattern sharpens monotonically with distance, consistent with the model using inter-cubie attention as a proxy for “how many cubies need to communicate about their positions.”

4.13 Attention Head Ablation Results

Head	Accuracy drop (pp)
L0H1	−44.9
L0H0	−30.7
L0H2	−25.1
L0H3	−9.3
L1H0–L3H3 (all 12)	≤ 7.7 each

Table 5: Accuracy drop from mean-attention ablation of individual heads in the corner model. Layer-0 heads dominate; all Layer 1–3 heads are dispensable.

The three top-ranked heads are all in Layer 0, and collectively they account for essentially all of the model’s inter-cubie routing. Ablating L0H1 alone drops accuracy by 44.9 pp—the

largest single-component effect of any experiment in this project. By contrast, ablating all 12 heads in Layers 1–3 simultaneously produces only minor degradation. This mirrors the flat-model result: just as `mlp_0` dominates the flat model’s distance circuit, the Layer-0 attention heads dominate the corner model’s routing circuit. In both architectures, the bulk of the meaningful computation is concentrated in the first transformer block.

5 Discussion: How Results Changed Our Thinking

The project began with the hypothesis that the transformer would learn a hierarchical representation—earlier layers encoding low-level features (individual sticker colors), later layers encoding higher-level structure (face solved status, distance to goal). The results tell a more nuanced story.

The linear embedding already encodes optimal distance nearly completely. This is not because the task is easy—78.5% accuracy on a 12-class problem is non-trivial—but because the one-hot encoding of a cube state implicitly contains all distance information in a linearly separable form. The 144-dim one-hot vector is highly redundant (physical cube states satisfy hard constraints, so most 144-dim vectors are invalid), and the BFS distance is a smooth function of the sticker configuration. The transformer learns to read this function linearly from the very first layer.

The transformer blocks then *rotate* the representation to align it with the unembedding direction, as evidenced by the logit/tuned-lens gap. They do not add new information but do meaningful work refining the basis. This is consistent with Variengien et al.’s prediction that belief states are linearly represented in residual streams [11]—our model is a direct instantiation of this, since the cube’s belief state for a single-token input is a point mass.

Circuit analysis sharpens this picture: virtually all the distance computation happens in `mlp_0`, with attention heads contributing negatively in aggregate. The circuit is nearly a single linear-nonlinear-linear pass. This makes the model unusually amenable to mechanistic analysis—the relevant computation is localized to one component in one layer.

The text representation experiments, run at scale across five models, produced a starker finding than the pilot suggested. While the one-case pilot hinted that `piece_identity`—by making permutation structure explicit—outperforms position-based representations, this did not survive replication: all state-based representations scored zero uniformly across every model and every distance level tested. The lesson is not that representation choice is irrelevant, but that the underlying capability—mentally simulating move sequences—is absent regardless of how the state is described. Move inversion (`move_sequence`), which requires no state understanding at all, separates the models cleanly: frontier models (GPT-5.x, Gemini 2.5 Pro) achieve 83–86%, while GPT-4o and Llama 3.3 70B reach only 12–16%.

The corner-tokenized architecture was introduced to give the model a natural multi-token structure for attention. Two results are notable. First, Layer-0 attention heads do nearly all the inter-cubie routing work—the same “first block dominates” pattern seen in the flat model. Second, the absence of superposition (confirmed by SAE analysis showing no dead features and continuously growing active counts with expansion ratio) means the model is not packing

more concepts than dimensions into its residual stream. These two findings together suggest the cube task does not push the model toward superposition because the task itself is simple enough to be solved without representational compression.

6 Remaining Work

1. **Scaled automated text-representation evaluation.** The current automated evaluation uses 10 cases per distance level. A 100-case run (in progress) will determine whether any state-based representation achieves non-zero solve rates, and will include `piece_identity` alongside the three representations already tested.
2. **Full corner-model circuit.** Head ablation identifies which Layer-0 heads are load-bearing, but does not explain *what* each head computes. Path-patching and DLA on the corner model would characterize the complete circuit from cubie token to distance logit, analogous to the flat-model circuit analysis.
3. **Scale to $3 \times 3 \times 3$.** The architecture and pipeline transfer directly. The $3 \times 3 \times 3$ cube has $\sim 4.3 \times 10^{19}$ states and God’s number 26, making BFS infeasible; a learned heuristic (e.g., value network trained on random-walk data) would replace the distance table. The key question is whether the “linear embedding encodes distance” finding generalizes, and whether superposition emerges at scale when the task is harder.

7 Roadblocks

Venv corruption. Bumping the package version from 0.1.0 to 0.2.0 without deleting the old `.venv` first caused a missing RECORD file warning and an incomplete install. Resolved by deleting and recreating the environment.

BFS orientation bias. The initial BFS was seeded from a single solved state, so the solver treated the 23 other valid orientations as non-solved. Solved by seeding from all 24 rotationally equivalent solved states; the regenerated distance table confirms 24 states at distance 0.

Installed-wheel path resolution. When installed as a wheel, `__file__` resolves to `site-packages`, not the project root, causing the visualizer’s solver panel to fail to locate `data/distances_cache.pkl`. Fixed by adding a `_find_project_root()` function that walks up from `os.getcwd()` looking for `pyproject.toml`.

Rate limiting in parallel API evaluation. Running five models in parallel via provider APIs triggered 429 rate-limit errors, particularly from Groq. Resolved by adding per-provider sleep intervals (10 s for Groq, 4 s for others) and exponential backoff with regex parsing of “retry-after” headers.

8 Questions

1. **The confabulatory CoT finding.** The chain-of-thought transcripts show the model claiming to verify its solutions without actually doing so. Is this a finding worth reporting as a standalone result (“pre-trained LLMs confabulate cube reasoning under CoT”), or is it too well-known a phenomenon to be worth space in the paper?
2. **Connecting to belief state geometry.** Variengien et al. [11] prove that optimal predictors encode belief states linearly. Our from-scratch model is an optimal predictor for a task with known ground truth—so in principle the theory applies directly. Is this connection strong enough to justify making it a central framing, or is it better treated as motivation rather than a formal claim?
3. **The 100% residual-swap flip rate.** This result (swapping the full residual stream at the embedding layer causes 100% distance-class flip) is striking but may be trivially explained by the fact that the one-hot encoding fully determines the distance. Is it still worth reporting, or does the trivial explanation make it uninformative?
4. **First-block dominance as a general principle.** Both the flat model (mlp_0 dominates DLA) and the corner model (Layer-0 heads dominate ablation) show that the first transformer block does essentially all the work. Is this a meaningful finding about how transformers solve classification tasks with small state spaces, or is it specific to the cube task’s structure?

Contributions

All code, experiments, and writing were produced by Cole McGuire. Claude Code (Anthropic) was used as a coding assistant for implementation and debugging throughout the project.

References

- [1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, “Solving the rubik’s cube with deep reinforcement learning and search,” *Nature machine intelligence*, vol. 1, no. 8, pp. 356–363, 2019.
- [2] K. Takano, “Self-supervision is all you need for solving rubik’s cube,” *arXiv.org*, 2023.
- [3] M. E. Chasmai, “CubeTR: Learning to solve the rubik’s cube using transformers,” *arXiv preprint arXiv:2111.03041*, 2021, withdrawn by the author.
- [4] P. Gupta, H. Conklin, S.-J. Leslie, and A. Lee, “Better world models can lead to better post-training performance,” *arXiv preprint arXiv:2512.03400*, 2024.
- [5] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” *arXiv.org*, 2016.

- [6] Y. Belinkov, “Probing classifiers: Promises, shortcomings, and advances,” *Computational linguistics - Association for Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2022.
- [7] N. Belrose, I. Ostrovsky, L. McKinney, Z. Furman, L. Smith, D. Halawi, S. Biderman, and J. Steinhardt, “Eliciting latent predictions from transformers with the tuned lens,” *arXiv.org*, 2025.
- [8] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in GPT,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [9] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLaughlin, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah, “Towards monosemanticity: Decomposing language models with dictionary learning,” *Transformer Circuits Thread*, 2023, <https://transformer-circuits.pub/2023/monosemantic-features>.
- [10] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, “Progress measures for grokking via mechanistic interpretability,” *arXiv preprint arXiv:2301.05217*, 2023.
- [11] A. Variengien, E. Winsor, T. Shearer, and Z. Hatfield-Dodds, “Transformers represent belief state geometry in their residual stream,” *arXiv preprint arXiv:2405.15943*, 2024.
- [12] nostalgebraist, “interpreting GPT: the logit lens,” <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/>, 2020.